34.1 Radix vs. Comparison Sorting

Intuitive Analysis

Merge Sort Runtime

Merge Sort requires Θ(N log N) compares.

A key concept here is that Merge Sort's runtime on strings of length W is Θ(N log N) if each comparison takes constant time or Θ(WN log N) if each comparison takes Θ(W) time.

Example of Θ(N log N): Strings are all different in top character.

Example of Θ(WN log N): Strings are all equal.

LSD vs. Merge Sort

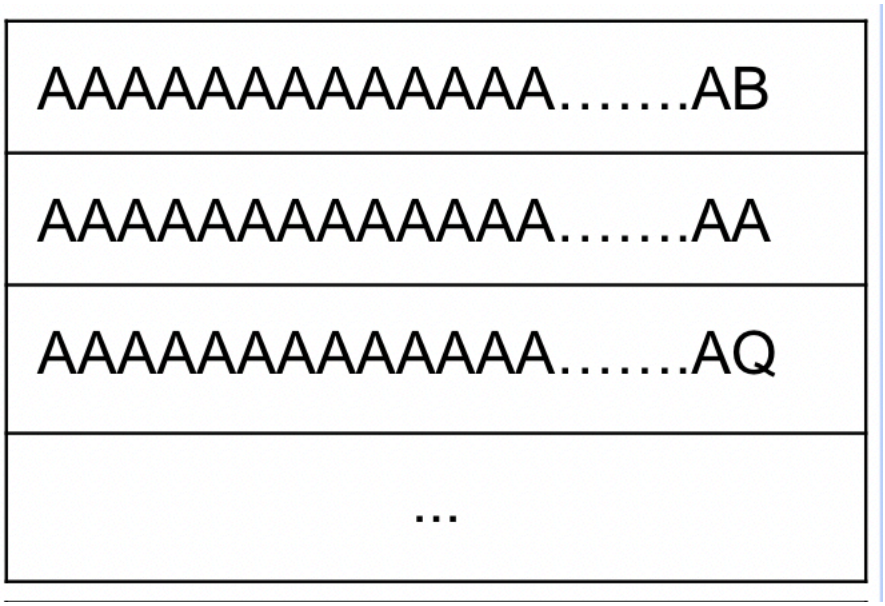
The facts:

Treating alphabet size as constant, LSD Sort has runtime Θ(WN). Merge Sort has runtime between Θ(N log N) and Θ(WN log N).

Which is better? It depends.

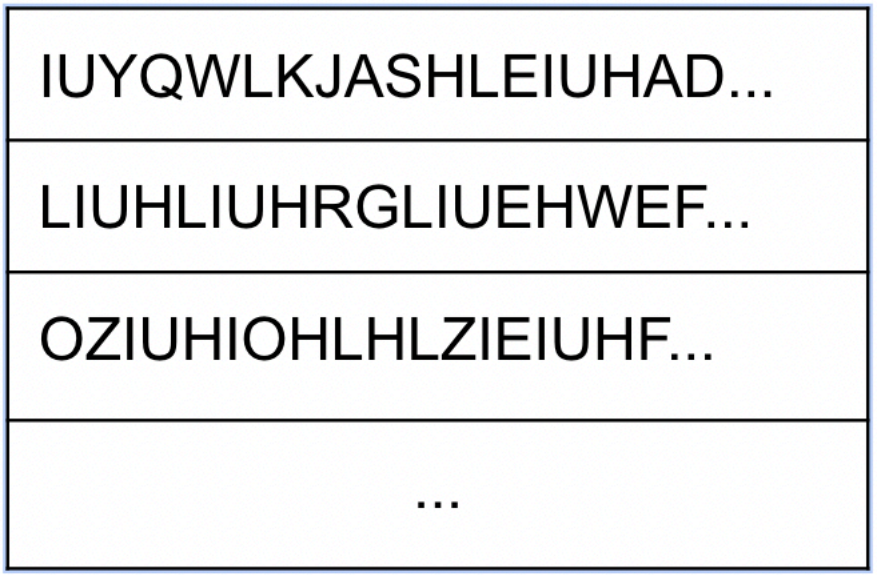
When might LSD sort be faster?

- When W is sufficiently smaller than N (or equivalently, N is really really big, or as you get to larger and larger numbers of strings, we expect LSD to pull ahead).
- Worst case strings for mergesort, the more similar they are, the more we expect LSD sort to do better.
- Sufficiently large N.
- If strings are very similar to each other.
    - Each Merge Sort comparison costs Θ(W) time.



When might Merge Sort be faster?

- Strings are different, especially at the beginning.
- If strings are highly dissimilar from each other
  - Each merge sort comparison is very fast



Cost Model Analysis

Alternate Approach: Picking a Cost Model

An alternate approach is to pick a cost model.

- We'll use number of characters examined.
- By "examined", we mean:
  - Radix Sort: Calling charAt in order to count occurrences of each character.
  - Merge Sort: Calling charAt in order to compare two Strings.

MSD vs. Mergesort

Suppose we have 100 strings of 1000 characters each.

For MSD Radix Sort, in the worst case (all strings equal), every character is examined exactly once. Thus, we have exactly 100,000 total character examinations.

AAAAAA

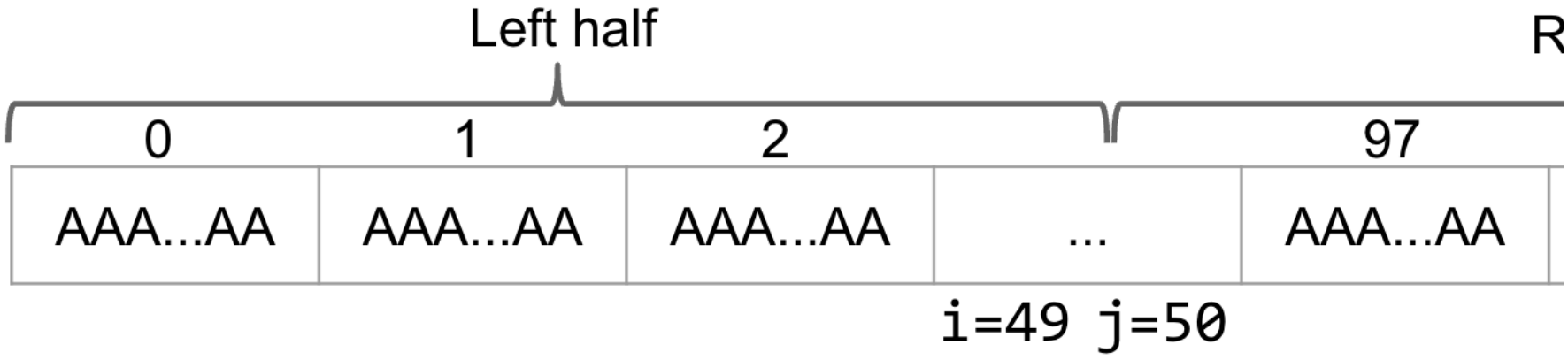
AAAAAA

AAAAAA

For Merge Sort, estimate the total number of characters examined if all strings are equal.

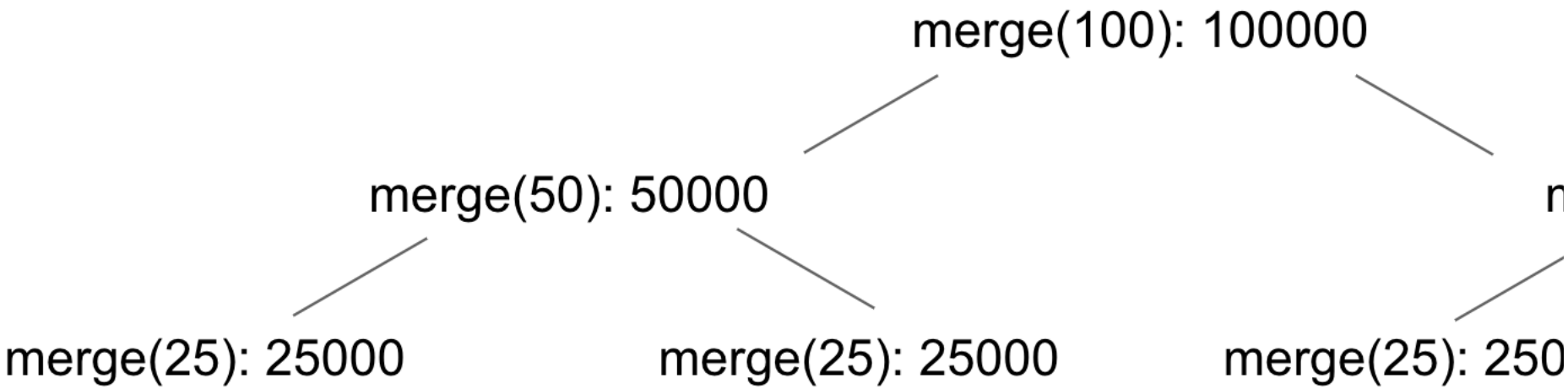
Merging 100 items, assuming equal items results in always picking left:

- Comparing A[0] to A[50]: 2000 character examinations.
- Comparing A[1] to A[50]: 2000 character examinations.
- … Comparing A[49] to A[50]: 2000 character examinations.
- Total characters examined: 50 * 2000 = 100000.
- Merging N strings of 1000 characters requires N/2 * 2000 = 1000N examinations.



In total, we must examine approximately 1000N log2 N total characters.

- 100000 + 50000*2 + 25000 * 4 + … = ~660,000 characters.



MSD vs. Mergesort Character Examinations

For N equal strings of length 1000, we found that:

- MSD radix sort will examine ~1000N characters (For N= 100: 100,000).

- Merge sort will examine ~1000Nlog2(N) characters (For N=100: 660,000).

  If character examination are an appropriate cost model, we'd expect Merge Sort to be slower by a factor of log2N.

  To see if we're right, we'll need to do a computational experiment.

Empirical Study: Radix Sort vs. Comparison Sorting

Computational Experiment Results (Your Answers)

Computational experiment for W = 100.

- MSD and merge sort implementations are highly optimized versions taken from our optional algorithms textbook.
- Does our data match our runtime hypothesis? No! Why not?
  - Maybe when MSD makes partitions, the divide and conquer cost is not accounted for.
  - Is mergesort optimized to do the timesort optimization? Unclear.
  - Maybe this is caching performance, Josh mentioned MSD has bad cache performance.
  - Maybe string comparison is not linear time somehow (keep in mind every string compare does the SAME thing. Repeated work on real machines these days tends to get optimized away).
  - Our cost model isn't representative of everything that is happening.
  - One particularly thorny issue: **The "Just In Time" Compiler.**

Merge

N	Runtime	# chars
10,000	0.05	13,801,600
100,000	0.26	170,780,800
1,000,000	0.87	2,013,286,400
10,000,000	13.8	23,757,632,000

MSD

N	Runtime	# chars
10,000	0.05	1,000,000
100,000	1.98	10,000,000
1,000,000	30.21	100,000,000
10,000,000	370.26	1,000,000,000

34.2 The Just-In-Time Compiler

Java's Just-In-Time Compiler secretly optimizes your code when it runs.

- The code you write is not necessarily the code that executes!
- As your code runs, the "interpreter" is watching everything that happens.
  - If some segment of code is called many times, the interpreter actually studies and re-implements your code based on what it learned by watching WHILE ITS RUNNING (!!).
    - Example: Performing calculations whose results are unused.
    - See this video if you're curious.

JIT Example

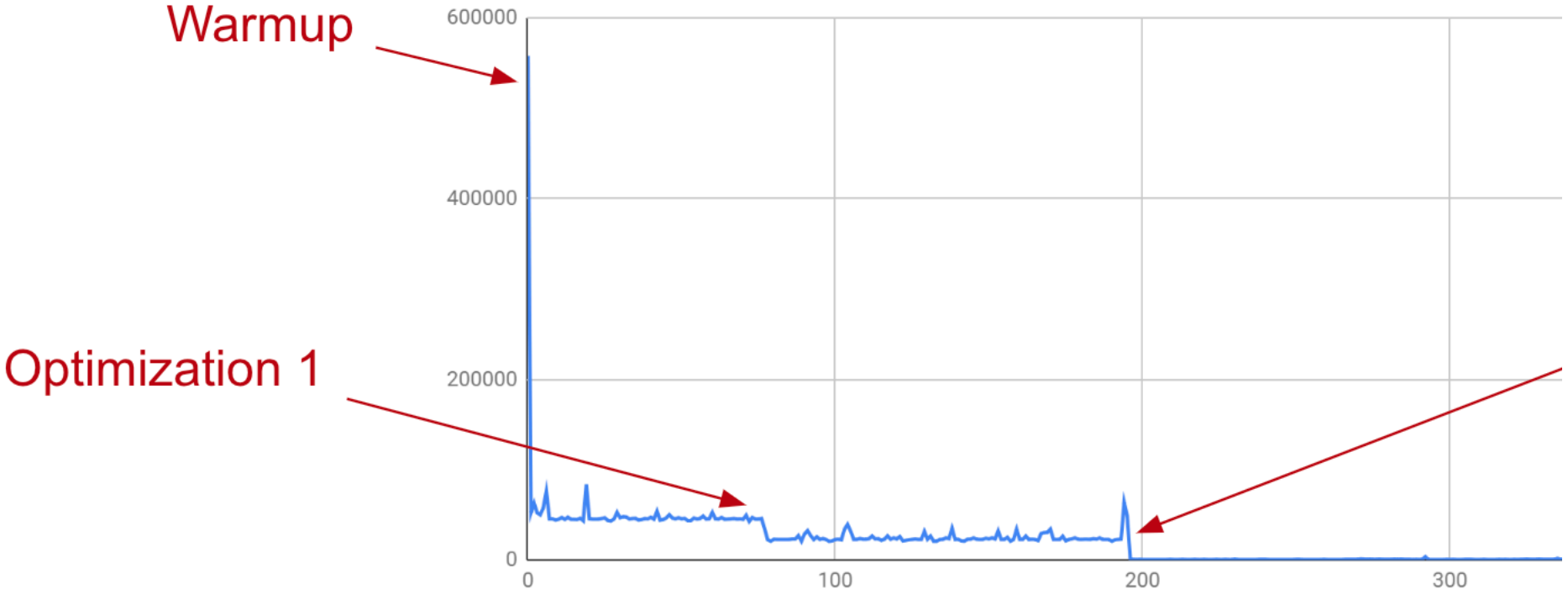
The code below creates Linked Lists, 1000 at a time.

- Repeating this 500 times yields an interesting result.

```
public class JITDemo1 {
   static final int NUM_LISTS = 1000;

   public static void main(String[] args) {
      for (int i = 0; i < 500; i += 1) {
         long startTime = System.nanoTime();
         for (int j = 0; j < NUM_LISTS; j += 1) {
            LinkedList<Integer> L = new LinkedList<>()
         }
         long endTime = System.nanoTime();
         System.out.println(i + ": " + endTime - star
      }
   }
}
```

- First optimization: Not sure what it does.
- Second optimization: Stops creating linked lists since we're not actually using them.



… So Which is Better? MSD or MergeSort?

The performance of the merge sort algorithm is highly dependent on the presence of just-in-time (JIT) compilation. Specifically, when JIT is enabled, merge sort is faster in cases where the strings being sorted are equal, but slower when JIT is disabled. This suggests that merge sort is generally more effective for this specific case, given that JIT is typically enabled. However, there are numerous other scenarios to consider, including the sorting of almost equal strings, randomized strings, and real-world data from specific datasets. When assessing the effectiveness of merge sort for these alternative cases, it is important to conduct careful experimentation and profiling to determine which sorting algorithm is most suitable. The lectureCode repository provides code for running these experiments, allowing for more precise assessment of algorithm performance under various conditions. Ultimately, real-world applications will require a thorough analysis of different implementations on actual data in order to select the optimal algorithm for the task at hand.

Bottom Line: Algorithms Can Be Hard to Compare

Comparing algorithms that have the same order of growth can be a difficult task, as it requires conducting computational experiments to determine their relative performance. However, modern programming environments can introduce additional challenges to this process, as certain optimizations such as just-in-time (JIT) compilation in Java can impact the results of experiments. It is worth noting that even small optimizations to an algorithm can have a significant impact on its performance. For instance, a change to the Quicksort algorithm suggested by Vladimir Yaroslavskiy has been shown to provide notable improvements, as discussed briefly in the Quicksort lecture. Therefore, when comparing algorithms with similar growth rates, it is crucial to remain vigilant of potential optimizations and variations that may influence their performance.

JIT Compilers Are Always Evolving

The JIT (just-in-time) compiler is a highly complex and essential component of modern compilers, and is an active area of research and development in this field. However, the older JIT compiler known as C2 has become increasingly difficult to maintain and extend, with no major improvements implemented in recent years. The codebase for C2 is written in a specific dialect of C++, making it challenging for new engineers to understand and work with. As a result, the codebase is being abandoned in favor of newer and more maintainable alternatives. For individuals interested in this area of study, CS164 offers a course on compilers and there are opportunities for involvement in ongoing research. It is worth noting that the reasons for the improved performance of merge sort with the JIT are not yet fully understood, making it an interesting topic for further research.

34.3 Radix Sorting Integers

{% hint style="info" %} BTW: A 32-bit integer is a normal integer in Java and other programming languages that spans 4 billion values. This is something we learned in Hashing chapter 20 and is a 61C concept! {% endhint %}

Summary of Video above

Barack Obama gets a Google Interview question: What is the most efficient way to sort a million 32-bit integers? Obama says that he would recommend not using Bubble sort! That's right folks! Obama knows his 61B :smile:.

What's the answer?

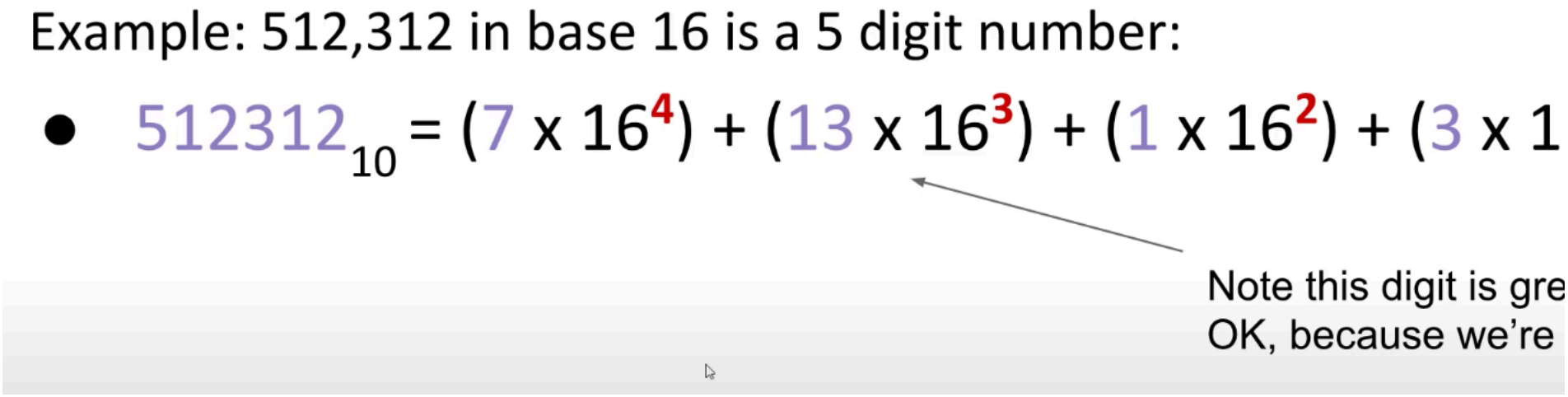
The answer to this question actually is Radix sort because we know that in a very large N limit, Radix sort is simply fastest as it is linear with runtime $\Theta(WN)$ where W is the number of digits and N is the number of integers we are sorting. This is much faster than any Comparison Sort we learned about since the fastest runtimes seem to be $\Theta(N*\log(N))$.

But how do we do it?

We don't have a charAt() for every integer. And if we converted all integers to strings, that's a very time expensive operation as well. So how would you LSD radix sort an array of integers? Instead of using charAt, maybe write a helper method like getDthDigit(int N, int d). Example: getDthDigit(15009, 2) = 5.

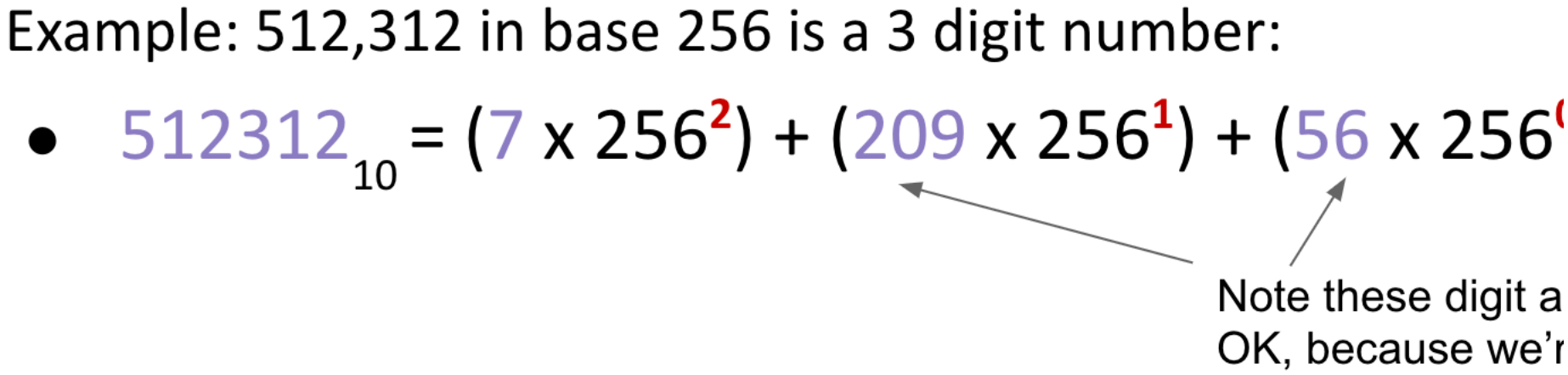
LSD Radix Sort on Integers

Note that we don't have to work with base 10. What we can instead do is increase this base to have less digits in our total number. This is really getting into 61C territory as we haven't really discussed binary representation of integers yet, but essentially what we want to do is convert to hexadecimal, a base 16 number that is represented with the following digits: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}. Note that A is 10 and F is 15. 0 - 15 is 16 possible values for a digit! To convert a number from decimal (base 10) to hexadecimal (base 16), we do the following:



Decimal to Hexadecimal

Notice that our original number had 6 digits in base 10 and our resulting number has 5 digits! Notice we don't have to go to just base 16. We can go even bigger to base 256 to only have 3 digits! There are small yet amazing optimizations!



Decimal to Ducentohexaquinquagesimal (Base 256)

Please do not worry about memorizing what "ducentohexaquinquagesimal" is. The only important thing to learn here is that this conversion to higher bases may result in fewer digits for our LSD and MSD Radix sorts to have faster traversals.

However, there is a tradeoff between W, the size of each integer, and the size of the array we need to maintain the counts. The most oprtimal is actually base 256!

34.4 Summary

Radix Sort vs. Comparison Sorts. In lecture, we used the number of characters examined as a cost model to compare radix sort and comparison sort. For MSD Radix Sort, the worst case is that each character is examined once for NM characters examined. For merge sort, MNlogN is the worst case characters examined. Thus, we can see that merge sort is slower by a factor of logN if character comparisons are an appropriate cost model. Using an empirical analysis, however, we saw that this does not hold true because of lots of background reasons such as the cache, optimized methods, extra copy operations, and overall because our cost model does not account for everything happening.

Just-In-Time Compiler. The "interpreter" studies your code as it runs so that when a sequence of code is run many times, it studies and re-implements based on what it learns while running to optimize it. For example, if a LinkedList is created many times in a loop and left unused, it eventually learns to stop creating the LinkedLists since they are never used. With the Just-In-Time compiler disabled, merge sort, from the previous section, is indeed slower than MSD Radix Sort.

Radix Sorting Integers. When radix sorting integers, we no longer have a charAt method. There are lots of alternative options are stilizing mods and division to write your own getDigit() method or to make each Integer into a String. However, we don't actually have to stick to base 10 and can instead treat the numbers as base 16, 256, or even base 65536 numbers. Thus, we can reduce the number of digits, which can reduces the runtime since runtime for radix sort depends on alphabet size.

34.5 Exercises

Factual

1. What happens when a Java file is compiled? HINT: Think about `.java` and `.class` files.
2. Why can a block of code run faster the second time it executes as compared to the first time?
3. What's the optimal way to sort an array of integers? Assume the integers are uniformly randomly distributed.

▶ Problem 1
▶ Problem 2
▶ Problem 3

Metacognitive

1. Under what conditions would LSD sort be faster than mergesort?

▶ Problem 1

34. Sorting and Data Structures Conclusion